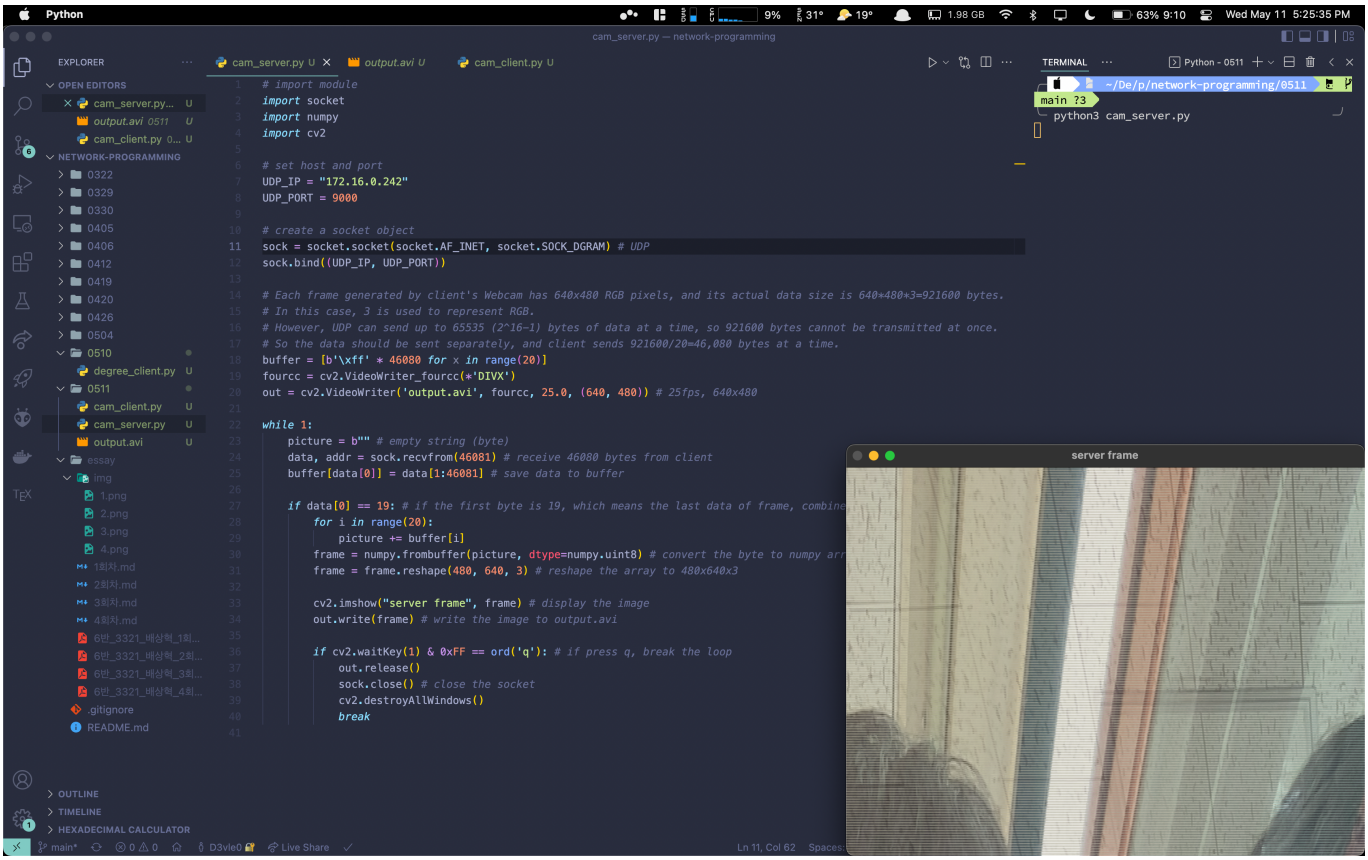
6반_3321_배상혁_5회차




```python
# 5회차
# import module
import socket
import numpy
import cv2

# set host and port
UDP_IP = "172.16.0.242"
UDP_PORT = 9000

# create a socket object
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # UDP
sock.bind((UDP_IP, UDP_PORT))

# Each frame generated by client's Webcam has 640x480 RGB pixels, and its
actual data size is 640*480*3=921600 bytes.
# In this case, 3 is used to represent RGB.
# However, UDP can send up to 65535 (2^16-1) bytes of data at a time, so
921600 bytes cannot be transmitted at once.
# So the data should be sent separately, and client sends 921600/20=46,080
bytes at a time.
# But in macOS, the size of UDP data that can be transmitted at once is
only 9126 bytes, which is smaller than 46080 bytes, so I need to enter the
following command to increase it to 65535 bytes.
# sudo sysctl -w net.inet.udp.maxdgram=65535
```

```
buffer = [b'\xff' * 46080 for x in range(20)]
fourcc = cv2.VideoWriter_fourcc(*'DIVX')
out = cv2.VideoWriter('output.avi', fourcc, 25.0, (640, 480)) # 25fps,
640x480

while 1:
    picture = b"" # empty string (byte)
    data, addr = sock.recvfrom(46081) # receive 46080 bytes from client
    buffer[data[0]] = data[1:46081] # save data to buffer

    if data[0] == 19: # if the first byte is 19, which means the last data
of frame, combines all the data in the list and display as an image.
        for i in range(20):
            picture += buffer[i]
        frame = numpy.frombuffer(picture, dtype=numpy.uint8) # convert the
byte to numpy array
        frame = frame.reshape(480, 640, 3) # reshape the array to
480x640x3

        cv2.imshow("server frame", frame) # display the image
        out.write(frame) # write the image to output.avi

        if cv2.waitKey(1) & 0xFF == ord('q'): # if press q, break the loop
            out.release()
            sock.close() # close the socket
            cv2.destroyAllWindows()
            break
```